WordPress Admin

1. An editor cannot upload images, but a colleague can. Give at least three likely causes. In what order would you check them, what evidence would you look for, and what would you do once the cause is confirmed?

   Answer:

   a) WordPress role or capability differences
      i. Evidence: compare the affected editor's role and the working colleague's role. Check whether the affected account has the upload_files capability using a trusted WordPress administration tool or a temporary diagnostic in a controlled environment.
      ii. Action: correct the role or capability assignment through the site's normal user-role configuration. Do not grant administrator access just to make uploads work.

   b) File size, file type, or server limits
      i. Evidence: check the exact upload error, file size, permitted MIME types, PHP upload limits, available disk space, and web server logs.
      ii. Action: correct the specific restriction that caused the failure. Do not increase limits indiscriminately or permit unsafe file types.

   c) Upload directory permissions or security controls
      i. Evidence: inspect the WordPress uploads directory ownership and permissions, storage errors, security plugin logs, and any web application firewall blocks. If the colleague succeeds on the same site, determine whether the failure follows the account or the uploaded file.
      ii. Action: fix incorrect ownership or permissions, or adjust a confirmed security rule through the appropriate administrator. Avoid setting every directory to 777

2. An article page is slow, particularly on mobile. Describe three improvements you would consider for a WordPress publishing site. For each, explain what you would measure first. Choose one improvement and explain how you would

check that it helped without breaking ads, analytics, or signed-in behaviour.

Answer:

a) Optimize images: this measures the image file sizes, image dimensions, LCP (Largest Contentful Paint).

b) Reduce JavaScript and third-party scripts: this measures the total JavaScript transferred, long tasks, INP (Interaction to Next Paint), script loading time.

c) Improve caching and server response: this measures the TTFB (Time to First Byte), cache hit rate, page response time, mobile performance.

The improvement I would choose

I would start with image optimization if measurements show that oversized images are contributing materially to the slow page. It is often a relatively contained change, but I would verify that the images actually responsible for poor LCP are being improved.

Testing procedure:

- ✧ Record baseline measurements using Lighthouse and Chrome DevTools under the same mobile test conditions.
- ✧ Identify the main article image and its current size, format, loading priority, and rendering time.
- ✧ Optimize the image and deploy the change to staging first.
- ✧ Test an anonymous article page, an editor's signed-in session, ad placements, analytics events, and responsive image rendering.
- ✧ Compare LCP, page weight, and visual rendering against the baseline.
- ✧ Check browser errors and network requests for missing images, duplicate analytics events, or broken ads.
- ✧ Roll back if the change materially worsens the user experience or breaks critical functionality.